# Contents

Contents . . . 1
Introduction . . . 1
Data Sensitivity Classification . . . 1
Input Validation . . . 2
Data Handling and Storage . . . 2
MySQL Access & Credential Security (new in Version 1.2) . . . 3
External Request Security . . . 3
Result Cache Security . . . 3
IPO Auto-Detection Security . . . 4
CAPTCHA-Solving Service Considerations . . . 4
Application-Level Security . . . 4
Future Access Control Model . . . 4
Audit Logging (Current and Future) . . . 5
Risk Summary . . . 5
Compliance Considerations . . . 6

Security & Access Control Document IPO Allotment Verification System Prepared by: Bhavya Bulani Version 1.2 · July 2026

## Contents

- Introduction
- Data Sensitivity Classification
- Input Validation
- Data Handling and Storage
- MySQL Access & Credential Security (new in Version 1.2)
- External Request Security
- Result Cache Security
- IPO Auto-Detection Security
- CAPTCHA-Solving Service Considerations
- Application-Level Security
- Future Access Control Model
- Audit Logging (Current and Future)
- Risk Summary
- Compliance Considerations

## Introduction

This document defines the security posture and access control approach for the IPO Allotment Verification System. Because the system processes PAN numbers and other personally identifiable client data, and because it interacts with external third-party registrar websites, security considerations span data handling, input validation, external-request behavior, database access, and — for future versions — authentication and authorization.

Version 1.2 replaces the SQLite-based data layer with MySQL, which introduces a new area of consideration: the system now depends on a networked database server with its own credentials, access controls, and connection-security posture, rather than a single local file. User login with role-based access control remains intentionally deferred to a future version.

## Data Sensitivity Classification

| Data Element | Sensitivity | Notes |
|---|---|---|
| PAN Number | High | Government-issued tax identifier; treat as PII |
| Client Name | Medium | Personally identifying in combination with PAN/Client Code |
| Client Code / BOID / DP ID | Medium | Broker/demat-specific identifiers |
| Allotment Status | Low–Medium | Not sensitive alone, but tied to identifiable clients |
| Uploaded Excel Files | High | May contain any of the above in bulk |
| Cached Allotment Results | Low–Medium | Same sensitivity as a live result; sensitivity does not decrease just because the data is cached |
| Auto-Synced IPO Data | Low | Public IPO metadata; not personally identifiable |
| Run Logs | Low | Operational metadata; should not include raw PAN values in plain log text |
| MySQL Credentials | High | Database username/password and connection string; compromise exposes all of the above |

## Input Validation

- PAN format validation: exactly 10 characters — 5 letters, 4 digits, 1 letter — validated with a strict pattern before any external submission.
- Client Code validation: validated against the applicable broker-specific format.
- No unvalidated identifier is ever transmitted to a registrar website. This applies regardless of how many IPOs are selected for a run.
- File type validation: only .xlsx and .xls are accepted.
- File size / row limits: uploads are capped at 10,000 rows.
- Column auto-detection is validated, not trusted blindly.
- IPO selection validation: every IPO ID submitted must correspond to an existing, `validated = 1` IPO record.
- All parameters passed to the MySQL data-access layer are additionally validated at the ORM/model level (type and length constraints matching the schema), providing defense in depth even though the frontend and API layers validate first.

## Data Handling and Storage

- PAN numbers and client identifiers stored in MySQL should be treated as sensitive fields: access to the database instance should be restricted to the application's service account, not shared broadly.
- Uploaded Excel files should be processed and then either securely deleted or stored in a restricted-access location.
- Exported result files should be treated with the same sensitivity as the source upload.
- Cached allotment results stored in MySQL carry the same sensitivity as live results and are subject to the same access restrictions.

- Run logs must not include raw PAN values or client names in free-text log lines; logs should reference internal record IDs instead.
- Data at rest in MySQL should rely on the database server's standard file-system and backup protections; where the hosting environment supports it, enabling MySQL's transparent data encryption (or full-disk encryption at the host/volume level) is recommended for the data directory.

## MySQL Access & Credential Security (new in Version 1.2)

Moving to a networked database server introduces access-control surface area that a single-file SQLite database did not have:

- **Dedicated application account**: the application connects to MySQL using a dedicated database user (not `root`), granted only the privileges it needs (`SELECT`, `INSERT`, `UPDATE`, `DELETE` on the application's own schema) — no `GRANT`, `DROP DATABASE`, or administrative privileges.
- **Credential storage**: MySQL host, port, username, password, and database name are supplied via environment variables or a secrets file excluded from version control (e.g., `.env`, listed in `.gitignore`) — never hard-coded in source or committed to the repository.
- **Network exposure**: the MySQL server should not be exposed to the public internet; it should be reachable only from the application's own host/network (e.g., bound to localhost, a private VPC, or protected by a firewall/security group allowing only the application server's IP).
- **Encrypted connections**: the application connects to MySQL over TLS where the hosting environment supports it, rather than an unencrypted connection, particularly if the application and database do not sit on the same physical/virtual host.
- **Connection pooling limits**: the connection pool size is capped at a sane multiple of the configured worker concurrency, both to protect MySQL from connection exhaustion and to limit the blast radius of a runaway process.
- **Least-privilege for auxiliary tooling**: any ad hoc admin/reporting access to the database (e.g., for a developer inspecting data during testing) uses a separate, narrowly-scoped read-only account rather than the application's write-capable credentials.
- **Backups**: regular MySQL backups (e.g., `mysqldump` or a managed provider's automated backup feature) are configured, and backup files — which contain the same sensitive PAN data as the live database — are subject to the same access restrictions as the live database itself.

## External Request Security

- Rate limiting / pacing: requests to any single registrar must be paced conservatively. This limit is enforced globally across a run.
- Timeouts: every registrar request must have a bounded timeout.
- Isolation of automation logic: browser-automation modules run in a sandboxed context per registrar.
- No credential storage for registrar sites.
- HTTPS only: all outbound requests to registrars and the official IPO data source must use HTTPS.

## Result Cache Security

- Cache poisoning prevention: only results produced by a completed, successful registrar check are eligible to be written to the cache.

- TTL enforcement: cache entries must be treated as expired the moment their `cache_expires_at` timestamp passes; the expiry check happens at read time in the MySQL query, not only via a background purge job.
- Cache scope: cache entries are keyed strictly to a client/IPO pair, using the `client_id` and `ipo_id` foreign keys rather than raw PAN string matching.
- Transparency to the user: a cached result must always be visibly labeled as such.
- No new external exposure: the cache is purely an internal optimization backed by MySQL; it does not introduce any new external endpoint.

## IPO Auto-Detection Security

- Source trust boundary: data returned by the external IPO data source is treated as untrusted input.
- Validation before exposure: a synced IPO record is only marked `validated = 1` after passing basic sanity checks; malformed records are logged but never surfaced.
- HTTPS only for the external IPO data source.
- No write-back: the sync job only reads from the external source.
- Rate-limited polling.
- Auditability: each sync run is logged.

## CAPTCHA-Solving Service Considerations

- Third-party data exposure: only the CAPTCHA challenge image itself is ever sent to an optional solving service — never client-identifying data (PAN, name, Client Code).
- Terms-of-use compliance: before enabling a third-party solver for any registrar, an administrator should confirm it does not conflict with that registrar's terms of use.
- Disabled by default.
- Provider isolation behind the common `CaptchaProvider` interface.
- No credential or cost surprises: enabling a solver provider is an explicit organizational decision.

## Application-Level Security

Given that Version 1.2 does not yet include authentication, the following interim controls apply:

- The application should be deployed within a trusted internal network only, not exposed to the public internet, until authentication is implemented.
- All API endpoints should validate and sanitize input server-side.
- File uploads must be scanned for basic structural validity before being parsed.
- Standard web security practices apply throughout: **parameterized queries via the SQLAlchemy ORM (no raw, string-concatenated SQL against MySQL)** to prevent SQL injection, and output encoding in the frontend to prevent script injection.
- Database migrations (Alembic) are reviewed before being applied to any shared environment, since a migration runs with elevated schema-modification privileges.

## Future Access Control Model

The roadmap calls for role-based access control as the next major enhancement.

| Role | Permissions |
|---|---|
| Admin | Manage users, manage registrar configuration, manage IPO auto-detection settings and cache TTL, view all run history and logs, full system configuration |
| Operator | Upload files, select single or multiple IPOs, run checks, view own and shared run history, download reports |
| Viewer (optional future role) | View dashboards and history, no ability to trigger new checks |

Planned authentication approach:

- Username/password or SSO-based login, with sessions or token-based authentication (e.g., JWT) for the API.
- Every check run and report download attributed to the authenticated user, feeding into per-user audit logs.
- Role checks enforced server-side on every relevant endpoint.
- Once implemented, application-level roles will be distinct from MySQL database-level accounts — end users will never connect to MySQL directly, only through the application's own authenticated API.
- Until this is implemented, all users of the internally-deployed Version 1.2 system are treated as having equivalent access, which is why the trusted-network deployment constraint above remains mandatory.

## Audit Logging (Current and Future)

- Current (Version 1.2): run-level logs capture start/end time, registrars used, cache-hit count, and success/failure/timeout counts, supporting operational troubleshooting. IPO sync events are similarly logged.
- Future: once authentication is added, logs will additionally capture which user initiated each run, which user triggered a manual IPO sync, and which user's request served a cached result.

## Risk Summary

| Risk | Mitigation |
|---|---|
| PAN data exposure via logs or exports | Exclude raw PAN from free-text logs; restrict access to exported reports |
| Unauthorized access to the tool (pre-authentication) | Restrict deployment to a trusted internal network for Version 1.2 |
| Registrar site blocking due to aggressive automation | Conservative rate pacing, per-registrar timeouts, fallback chain, result caching |
| Malformed or malicious file uploads | Strict file-type/row-count checks and structural validation before parsing |
| Injection via user-supplied data | Parameterized queries via the SQLAlchemy ORM; output encoding in the frontend |
| Stale or misleading cached results | Conservative TTL, explicit "served from cache" labeling, error states never cached |
| Malformed or untrustworthy auto-synced IPO data | Validation gate before an IPO becomes selectable; sync activity logged |

| Risk | Mitigation |
|---|---|
| Multi-IPO runs increasing registrar request volume | Global (not per-IPO) rate limiting; caching reduces duplicate load |
| Compromised or overly-broad MySQL credentials | Dedicated least-privilege application account; credentials outside version control; network access restricted to the application host |
| MySQL server exposed to the public internet | Bind to private network/localhost only, or firewall to the application server's IP; TLS for connections that cross a network boundary |

## Compliance Considerations

- PAN is a government-issued identifier in India and should be handled consistent with applicable data protection expectations for financial and personally identifiable information. This applies equally to cached PAN-linked results.
- Automated interaction with registrar websites should remain within each site's published terms of use.
- Storing PAN and client data in a networked MySQL instance (rather than a local SQLite file) makes database access control and credential hygiene a more central compliance concern than it was previously — this document's MySQL Access & Credential Security section above should be treated as a minimum baseline, not an exhaustive checklist, for any deployment handling real client data.